Class Definitions Demystified

Where you and Bill tread familiar ground: the Bookworm application and the Ruby object model.

You stumble sleepily into the office, craving your Thursday morning coffee, only to be ambushed by an excited Bill. "Hey!" he says. "Everyone likes the refactorings of Bookworm we did Monday, and the boss wants more. But before we start, let's go over some theory about class definitions. We'll begin where we left off Monday: in the Ruby object model."

Inside Class Definitions

You probably think of a class definition as the place where you define methods. In fact, you can put any code you want in a class definition:

```
class MyClass
  puts 'Hello'
end
```

```
Hello
```

Class definitions also return the value of the last statement, just like methods and blocks do:

```
result = class MyClass
  self
end

result # => MyClass
```

This last example emphasizes a compelling point that you might remember from *The self Keyword*, on page 34: in a class (or module) definition, the class itself takes the role of the current object `self`. Classes and modules are just objects, so why couldn't a class be `self`? Keep this point about class definitions and `self` in mind, because the concept will become useful a bit later.

While we're on the topic of `self`, you can learn about a related concept: that of the current class.

The Current Class

As you know, wherever you are in a Ruby program, you always have a current object: `self`. Likewise, you always have a *current class* (or module). When you define a method, that method becomes an instance method of the current class.